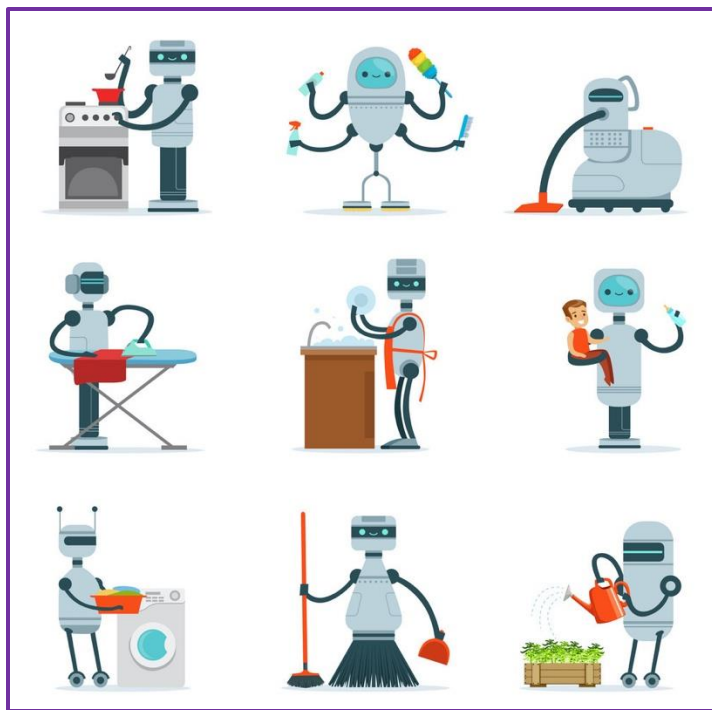


16-831: Introduction to Robot Learning, Fall 2024

Multi-Task and Adaptive Robot Learning



Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Project Logistics

- ☐ All logistics (schedule of pres, slides, final report) will be released tonight on Piazza
- ☐ Please read it carefully!

Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☐ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☒ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Outline

- ❑ What is multi-task robot learning and adaptive robot learning?
 - Task-wise adaptation
 - Dynamics-wise adaptation

How to adapt?

- ❑ (Classic) adaptive control
- ❑ Meta-learning and meta-RL
- ❑ Policy distillation (teacher-student, etc), end-to-end adaptation
- ❑ Goal-conditioned RL (GCRL)

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

- ❑ What is multi-task robot learning and adaptive robot learning?
 - Task-wise adaptation
 - Dynamics-wise adaptation

How to adapt?

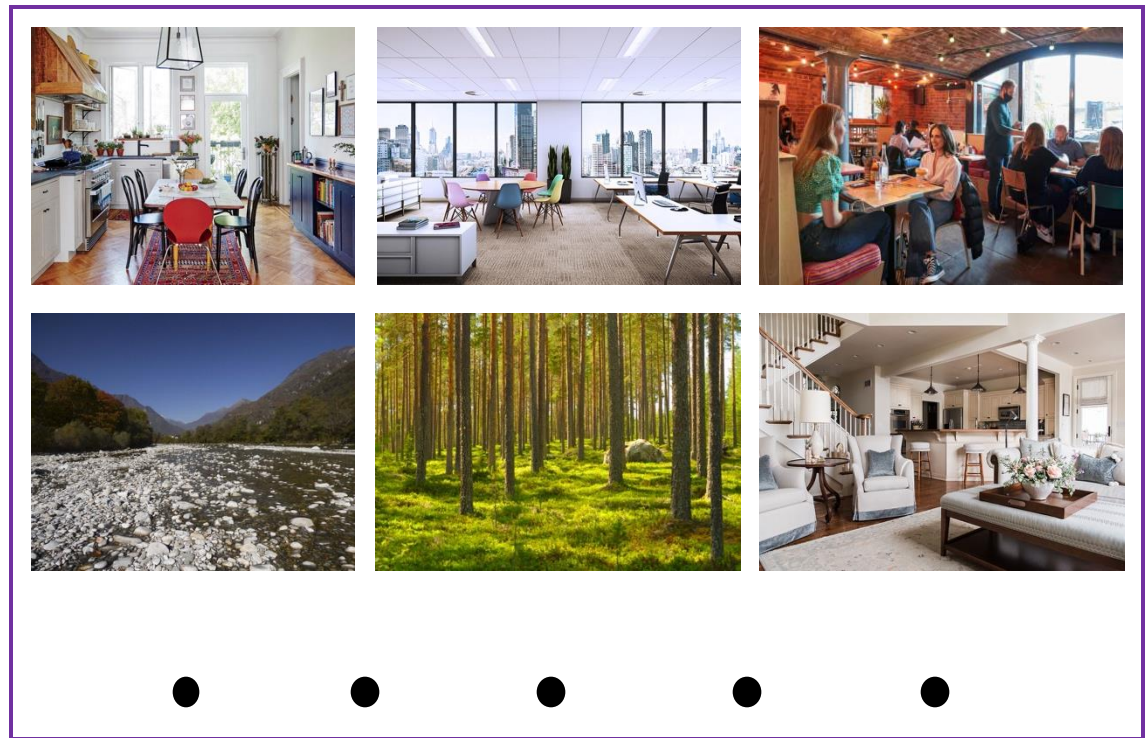
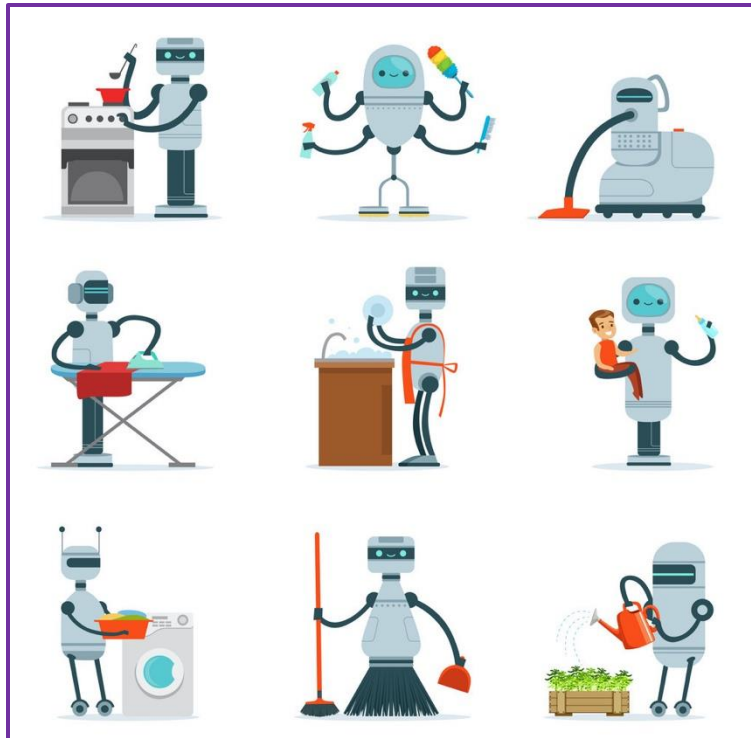
- ❑ (Classic) adaptive control
- ❑ Meta-learning and meta-RL
- ❑ Policy distillation (teacher-student, etc), end-to-end adaptation
- ❑ Goal-conditioned RL (GCRL)

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Motivation: Why Multi-Task and Adaptation?

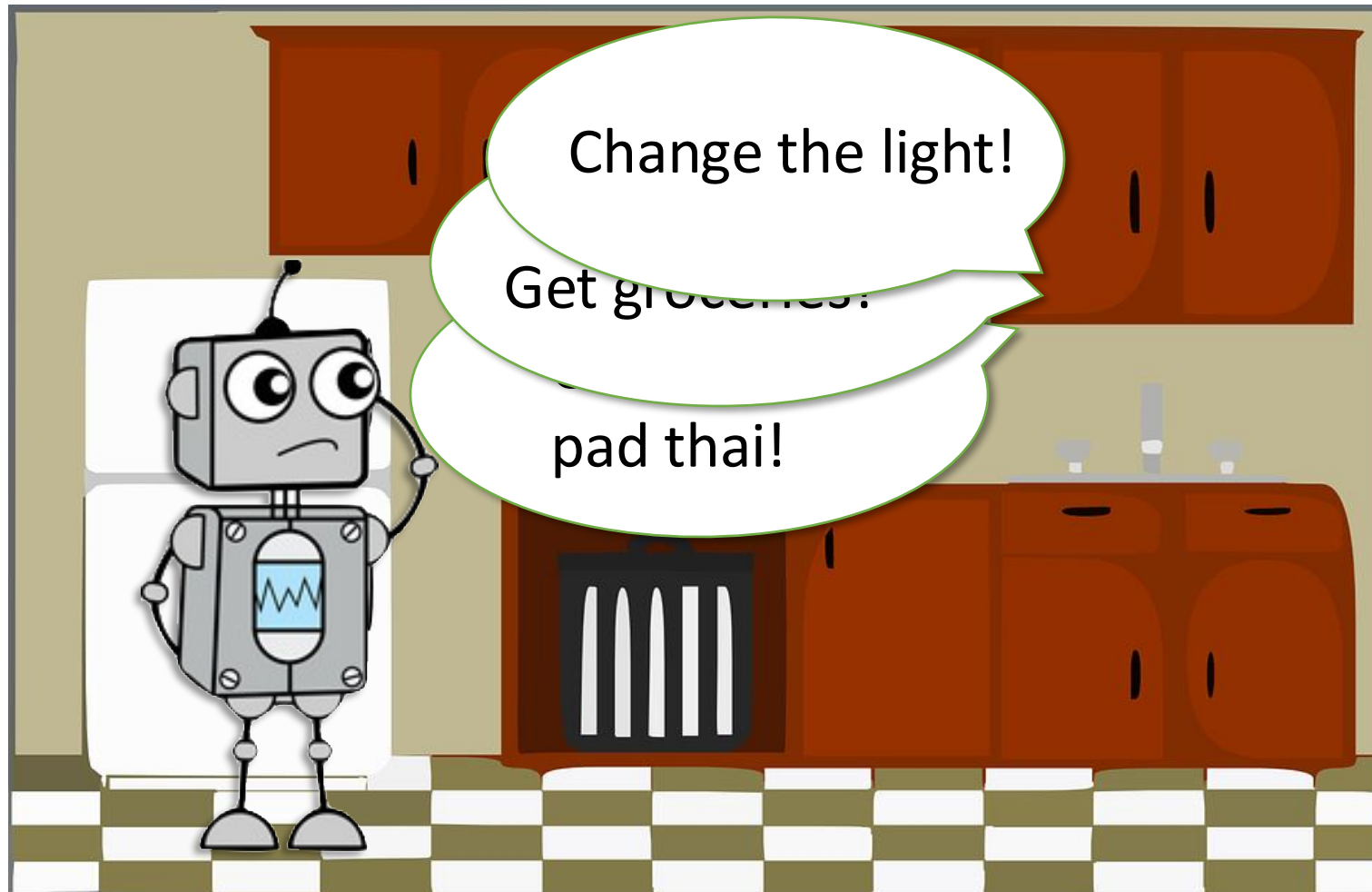
- ❑ Ultimate goal of robot learning: Build **general-purpose embodied** intelligence by learning to make sequential decisions in the physical world

Humans can do thousands of tasks in thousands of environments. How about robots?



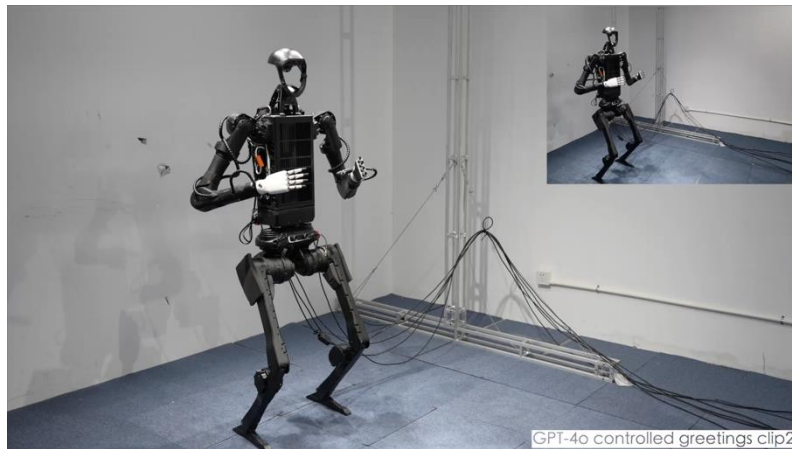
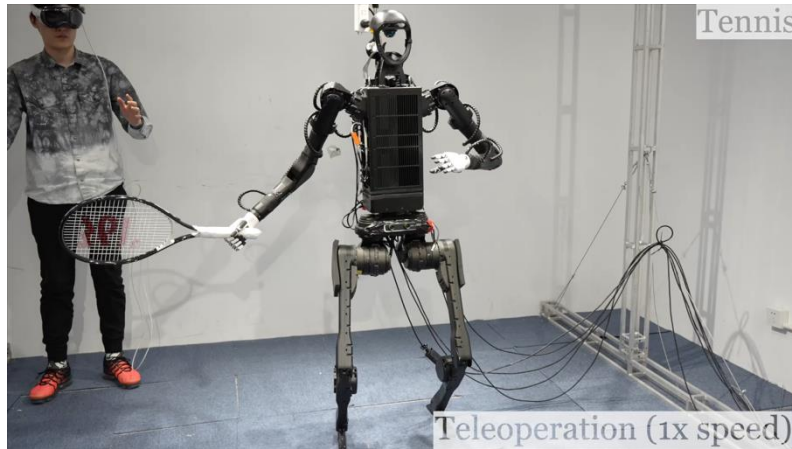
Motivation: Why Multi-Task and Adaptation?

- ❑ Ultimate goal of robot learning: Build **general-purpose embodied** intelligence by learning to make sequential decisions in the physical world



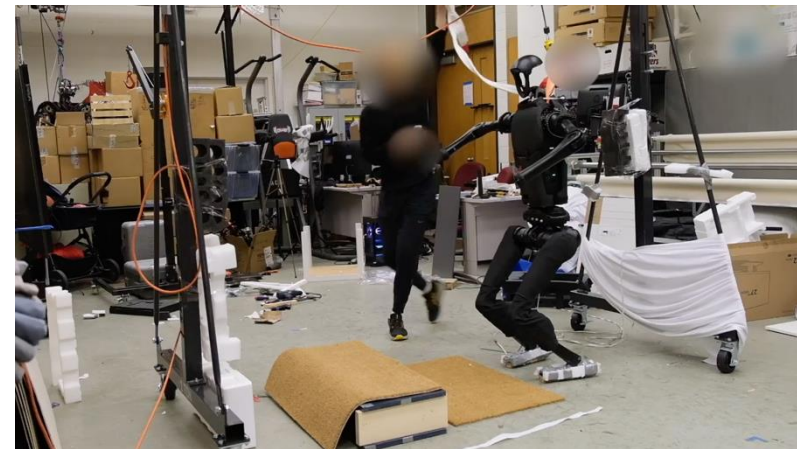
What is General-Purpose Robot?

- ❑ Robots that can do many different tasks in many different environments



OmniH2O

<https://omni.human2humanoid.com/>



WoCoCo

<https://lecar-lab.github.io/wococo/>

What is General-Purpose Robot?

- ❑ Robots that can do many different tasks in many different environments



Two Types of Adaptation

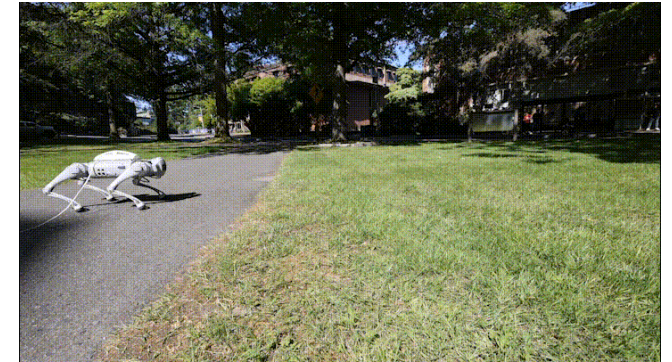
- ❑ *Dynamics-wise* (or *environment-wise*) adaptation
 - Robots operate in dynamic environments
 - The underlying dynamics model changes!

- ❑ *Task-wise* adaptation
 - One policy needs to handle different tasks
 - In particular, tasks unseen in training time

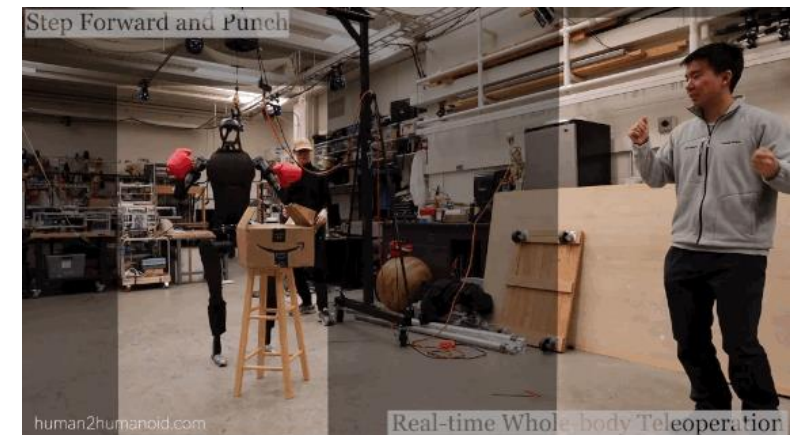
- ❑ These two types can co-exist!



Drone in a wind tunnel



Terrain conditions



One policy tracks many trajectories <https://human2humanoid.com/>

Dynamics-Wise Adaptation

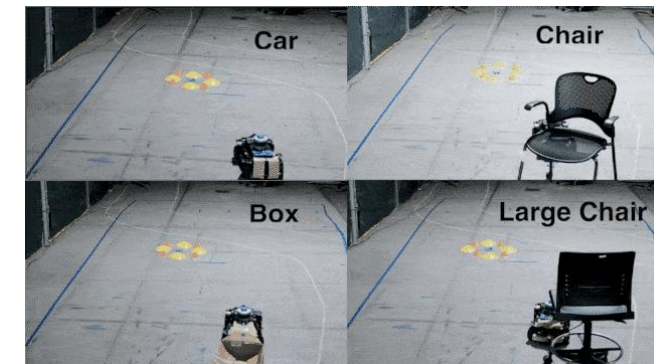
- ❑ Standard transition dynamics in an MDP: $s_{t+1} \sim P(\cdot | s_t, a_t)$
- ❑ Dynamics-wise adaptation: $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$
 - e_t is the environment parameter (e.g., wind, terrain, payload)
 - e_t might be time-variant
 - e_t is typically *unknown* in testing
- ❑ Objective 1: get an adaptive policy $a_t = \pi(s_t, z_t)$
 - z_t is the “context” variable
- ❑ Objective 2: design an adaptation law $z_t = A(\dots)$ such that $\pi(s_t, z_t)$ performs well under $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$



Drone in a wind tunnel



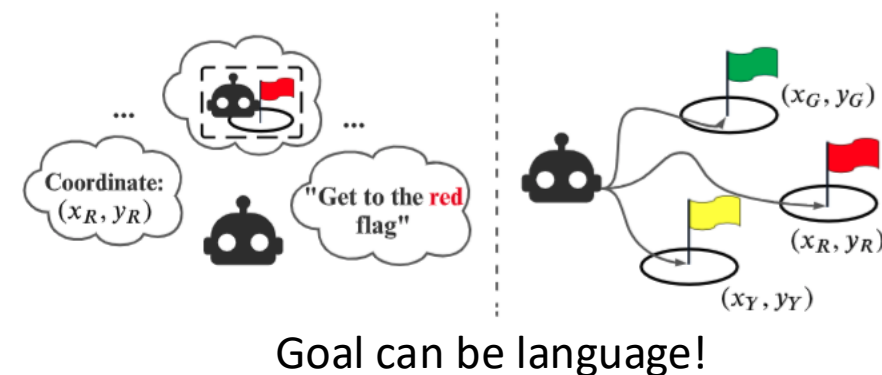
Terrain conditions



Payloads for a car

Task-Wise Adaptation

- ❑ Standard reward function in an MDP: $r(s_t, a_t)$
- ❑ Task-wise adaptation: $r(s_t, a_t, g)$
 - g is the task parameter (e.g., goal)
 - Also called “goal-conditioned RL”
 - g is often *known*, in both training and testing
 - However, g is often *unseen* in testing
- ❑ Objective: get an adaptive policy $a_t = \pi(s_t, g)$
 - $\pi(s_t, g)$ maximizes cumulative reward $r(s_t, a_t, g)$



Summary: Two Types of Adaptation

- ❑ *Dynamics-wise* (or *environment-wise*) adaptation
 - Design $a_t = \pi(s_t, z_t)$ and an adaptation law $z_t = A(\dots)$
 - Such that $\pi(s_t, z_t)$ performs well under $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$
- ❑ *Task-wise* adaptation
 - Design $a_t = \pi(s_t, g)$
 - Such that $\pi(s_t, g)$ maximizes cumulative reward $r(s_t, a_t, g)$

Two different adaptation settings:

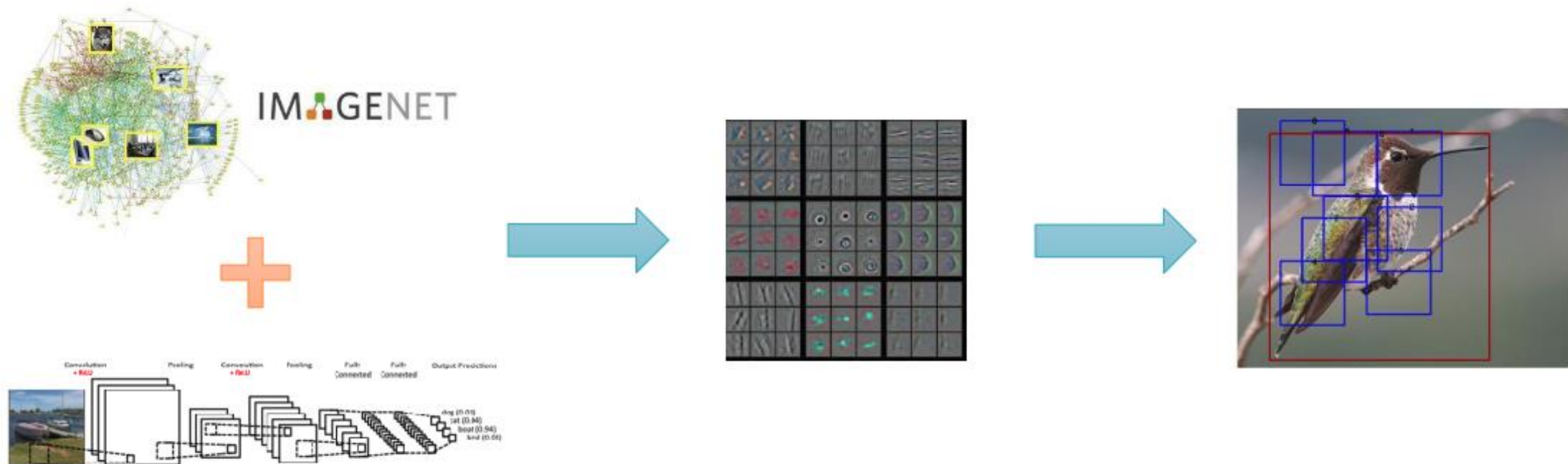
- ❑ *Zero-shot* adaptation: doesn't need labeled data in the adaptation process
- ❑ *Few-shot* adaptation: need (some) labeled data in the adaptation process

Remark: Another option is to learn a *single robust* policy $a_t = \pi(s_t)$ that works well for many e_t or g

- ❑ Already covered in previous lectures. We will NOT consider this setting today.

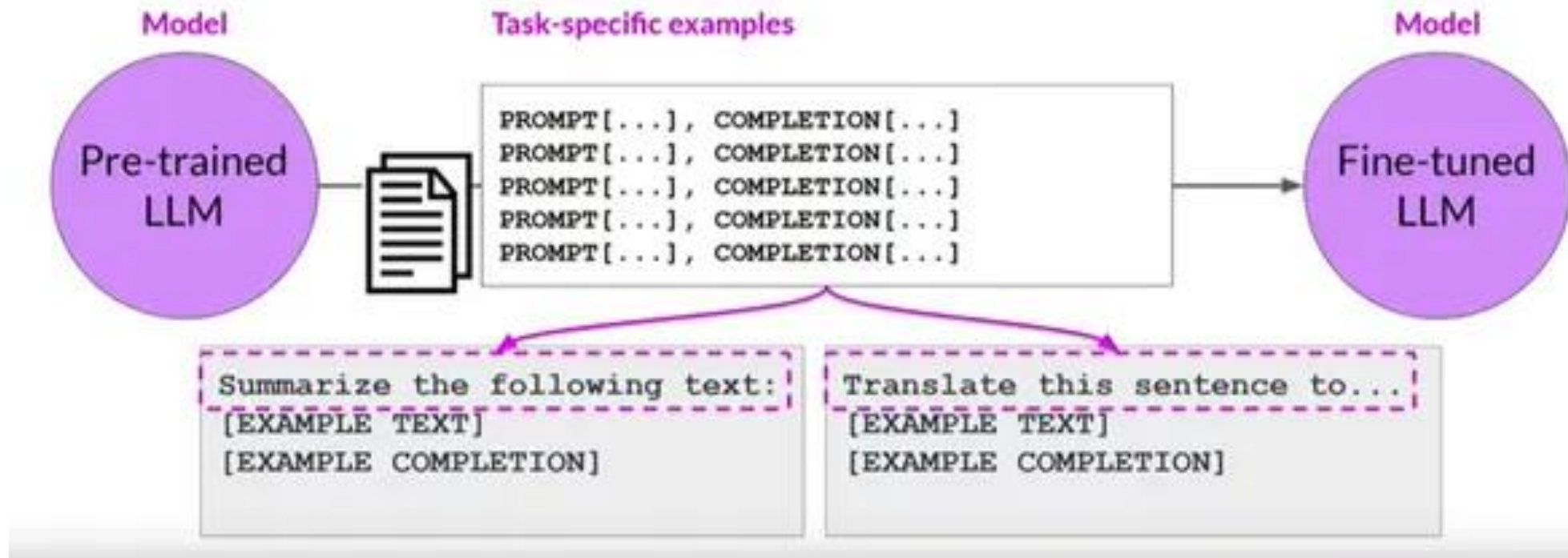
Lessons from Supervised Learning: Pretraining + Finetuning!

- ❑ The most successful paradigm in deep learning: pretraining + finetuning!
- ❑ Vision: pretraining visual features for downstream tasks



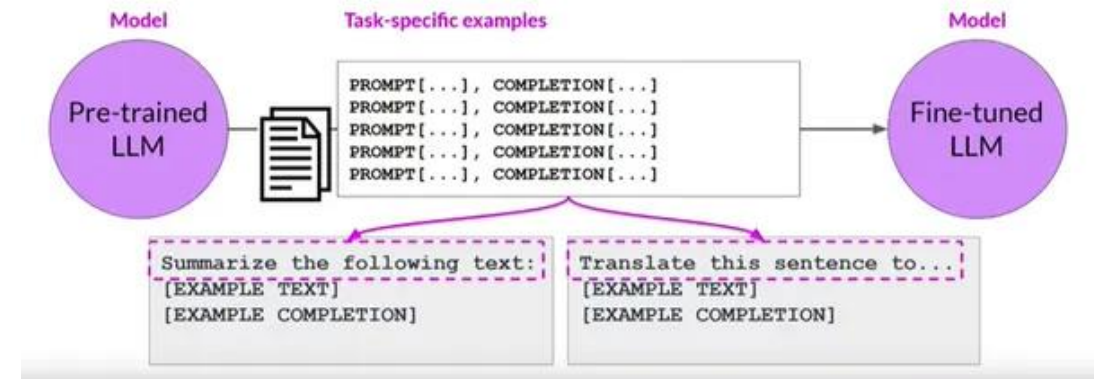
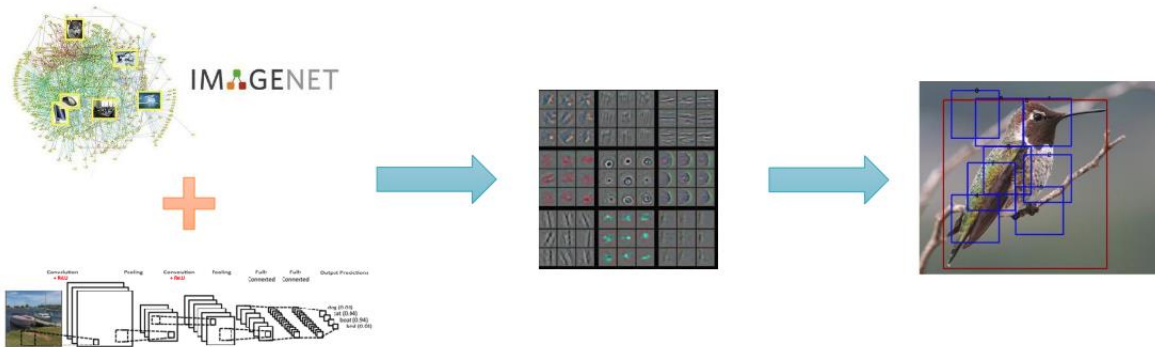
Lessons from Supervised Learning: Pretraining + Finetuning!

- ❑ The most successful paradigm in deep learning: pretraining + finetuning!
- ❑ Large language models: pretraining + instruction fine-tuning + RLHF



Lessons from Supervised Learning: Pretraining + Finetuning!

- ❑ The most successful paradigm in deep learning: pretraining + finetuning!
- ❑ Why does it work?
 - Different tasks share a common representation
 - Task-specific part lies in a low-dimensional manifold



Outline

- ❑ What is multi-task robot learning and adaptive robot learning?
 - Task-wise adaptation
 - Dynamics-wise adaptation

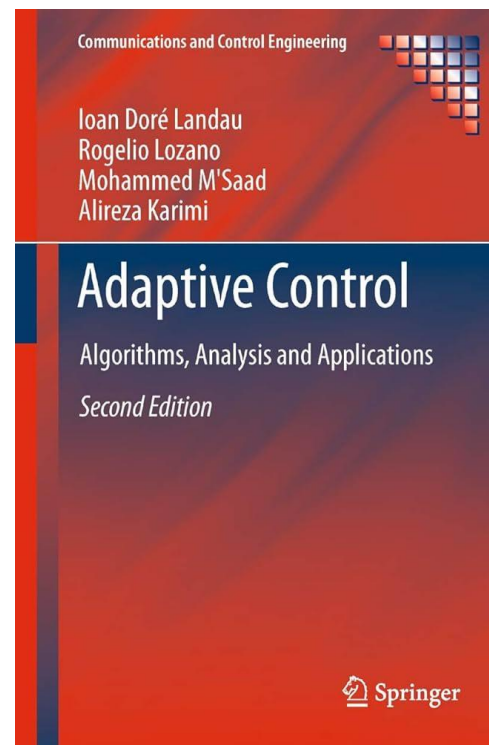
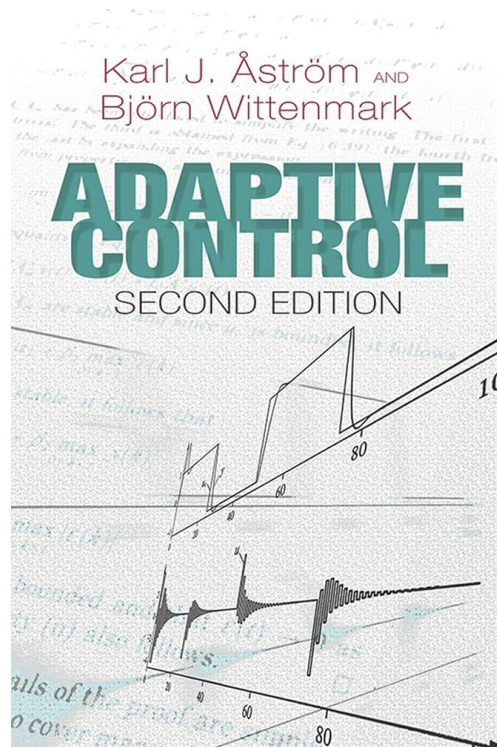
How to adapt?

- ❑ (Classic) adaptive control
- ❑ Meta-learning and meta-RL
- ❑ Policy distillation (teacher-student, etc), end-to-end adaptation
- ❑ Goal-conditioned RL (GCRL)

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Adaptive Control

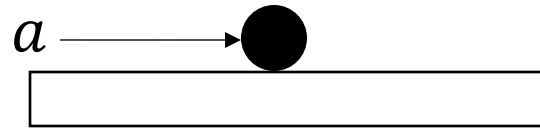
- ❑ Focus on *dynamics-wise* (or *environment-wise*) adaptation
- ❑ Using model-based control to design:
 - A policy $a_t = \pi(s_t, \hat{e}_t)$ and an adaptation law $\hat{e}_t = A(\dots)$
 - Such that $\pi(s_t, \hat{e}_t)$ performs well under $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$
- ❑ A very mature and big field in control!



Adaptive Control

❑ Using model-based control to design:

- A policy $a_t = \pi(s_t, \hat{e}_t)$ and an adaptation law $\hat{e}_t = A(\dots)$
- Such that $\pi(s_t, \hat{e}_t)$ performs well under $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$



A simple example: integral control (“I” in PID):

❑ Consider an uncertain dynamical system $\dot{s} = a + e$ where e is unknown

❑ Control objective: $s \rightarrow 0$

❑ P controller: $a = -2s$

- Closed-loop dynamics $\dot{s} = -2s + e$
- s will converge to $e/2$, NOT 0!

❑ PI controller: $a = -2s - \int_t s$

- Closed-loop system: $\dot{s} = -2s - \int_t s + e$
- s will converge to 0!
- $\int_t s$ can be viewed as $\hat{e}_t$. $\hat{e}_t = A(\text{history of } s)$.

Adaptive Control

- ❑ Using model-based control to design:
 - A policy $a_t = \pi(s_t, \hat{e}_t)$ and an adaptation law $\hat{e}_t = A(\dots)$
 - Such that $\pi(s_t, \hat{e}_t)$ performs well under $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$

Two types of adaptive controller:

- ❑ MIAC: model identification adaptive control (e.g., $\mathcal{L}_1$ adaptive control)
 - Perform system ID while the system is running
 - Goal: $\hat{e} \rightarrow e$ and some regularizations
 - More intuitive, but stability and robustness are tricky
- ❑ MRAC: model reference adaptive control (e.g., integral control)
 - Incorporate an “ideal” model defining desired closed-loop performance
 - MRAC does NOT directly do system ID
 - $\hat{e} \rightarrow e$ is neither necessary nor important

Outline

- ❑ What is multi-task robot learning and adaptive robot learning?
 - Task-wise adaptation
 - Dynamics-wise adaptation

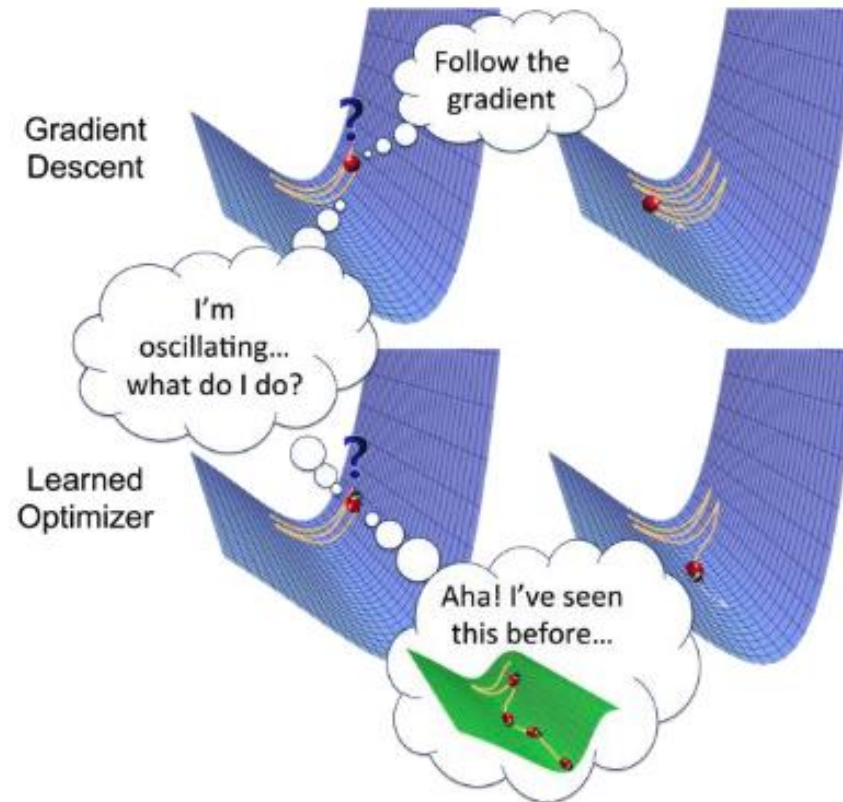
How to adapt?

- ❑ (Classic) adaptive control
- ❑ **Meta-learning and meta-RL**
- ❑ Policy distillation (teacher-student, etc), end-to-end adaptation
- ❑ Goal-conditioned RL (GCRL)

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Meta-Learning

- ❑ Meta-learning is *learning to learn*
- ❑ Example:
 - Standard supervised learning: gradient descent
 - Meta-learning optimizer: learn how to GD



Meta-Learning

- ❑ Generic supervised learning:

Learning parameter: θ

Training data: D^{tr}

Goal: $\min_{\theta} \mathcal{L}(\theta, D^{\text{tr}})$

- ❑ Generic meta-learning:

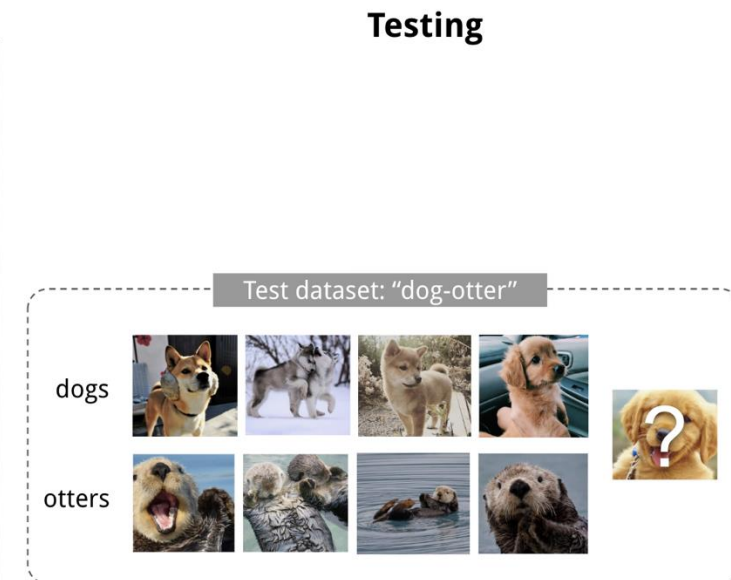
Meta dataset contains many tasks

Meta training data i : D_i^{tr}

Meta test data i : D_i^{ts}

Goal: $\min_{\theta} \sum_i \mathcal{L}(\phi_i, D_i^{\text{ts}})$

where $\phi_i = f_{\theta}(D_i^{\text{tr}})$



"Our training procedure is based on a simple machine learning principle: test and train conditions must match"
- Vinyals et al., Matching Networks for One-Shot Learning

Meta-Learning Example: Learning an Optimizer

- ❑ Meta-learning has many formulations

Meta dataset contains many tasks

Meta training data $i: D_i^{\text{tr}}$

Meta test data $i: D_i^{\text{ts}}$

Goal: $\min_{\theta} \sum_i \mathcal{L}(\phi_i, D_i^{\text{ts}})$

where $\phi_i = f_{\theta}(D_i^{\text{tr}})$

- ❑ One way to formulate: f_{θ} is a parametric optimizer
 - θ : hyperparameter of the optimizer
- ❑ Example: θ is the learning rate of an SGD optimizer
 - Goal: learn/find a good learning rate that works for many tasks

Meta-Learning Example: Representation Learning

- θ can also be viewed as a representation.

Example: Collect data $\{D_1, \dots, D_I\}$ from I tasks: $D_i = \{x_i^{(k)}, y_i^{(k)}\}_k$

- Representation learning or bi-level optimization:

$$\min_{h_\theta, \phi_1, \dots, \phi_I} \sum_{i=1}^I \sum_k \|y_i^{(k)} - h_\theta(x_i^{(k)}) \phi_i\|^2$$

- θ : parameter of the representation h_θ
- ϕ_i : linear coefficient in each task

Meta-Learning Algorithm: Black-Box Adaptation

- Key idea: train a neural network to represent $\phi_i = f_\theta(D_i^{\text{tr}})$
 - I.e., directly “predict” the weight ϕ_i in each task

Meta dataset contains many tasks

Meta training data i : D_i^{tr}

Meta test data i : D_i^{ts}

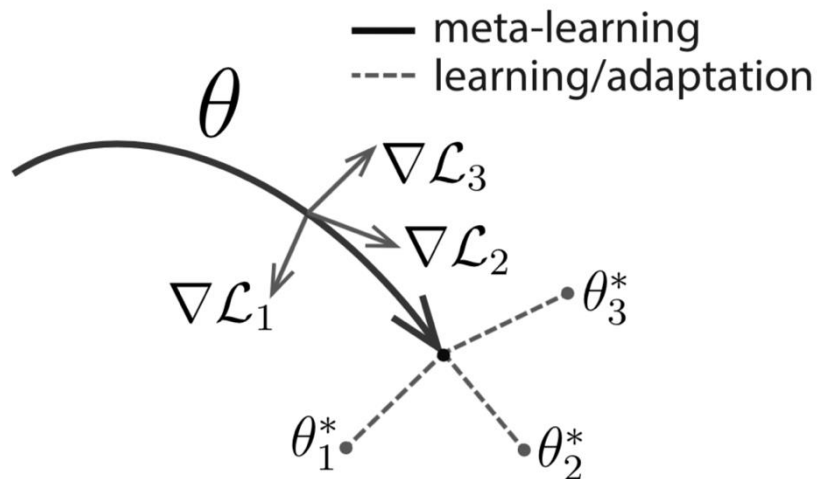
Goal: $\min_{\theta} \sum_i \mathcal{L}(\phi_i, D_i^{\text{ts}})$

where $\phi_i = f_\theta(D_i^{\text{tr}})$

1. Sample task $\mathcal{T}_i$ (or mini batch of tasks)
2. Sample disjoint datasets $\mathcal{D}_i^{\text{tr}}, \mathcal{D}_i^{\text{test}}$ from $\mathcal{D}_i$
3. Compute $\phi_i \leftarrow f_\theta(\mathcal{D}_i^{\text{tr}})$
4. Update θ using $\nabla_{\theta} \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{test}})$

Meta-Learning Algorithm: MAML

- ❑ Problem of black-box adaptation: it is not reliable to “guess” weights!
- ❑ We should leverage gradient information
- ❑ MAML: Model-Agnostic Meta-Learning (Finn et al., ICML’17)
- ❑ Key idea: ϕ_i is a one-step GD update of θ !
- ❑ Cons: Require Hessians



Algorithm 1 Model-Agnostic Meta-Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

1: randomly initialize θ

2: **while** not done **do**

3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$

4: **for all** $\mathcal{T}_i$ **do**

5: Evaluate $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ with respect to K examples

6: Compute adapted parameters with gradient descent: $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$

7: **end for** **Note:** the meta-update is using different set of data.

8: Update $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$ $\leftarrow \mathcal{L}_{\mathcal{T}_i}^{(1)}(f_{\theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}^{(0)}(f_{\theta})})$

9: **end while**

Meta-Learning Algorithms

- ❑ Many meta-learning algorithms!
- ❑ References:

Meta-Learning: Learning to Learn Fast

Date: November 30, 2018 | Estimated Reading Time: 30 min | Author: Lilian Weng

<https://lilianweng.github.io/posts/2018-11-30-meta-learning/>

Tutorial 16: Meta-Learning - Learning to Learn

Status **Finished**

Filled notebook: [Repo](#) [View On Github](#) [Open in Colab](#)

Pre-trained models: [Repo](#) [View On Github](#) [GDrive](#) [Download](#)

Recordings: [YouTube Part 1](#) [YouTube Part 2](#) [YouTube Part 2](#)

Author: Phillip Lippe

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial16/Meta_Learning.html

Meta-Learning: from Few-Shot Learning to Rapid Reinforcement Learning

ICML 2019 Tutorial

Meta-Learning + RL: Meta-RL

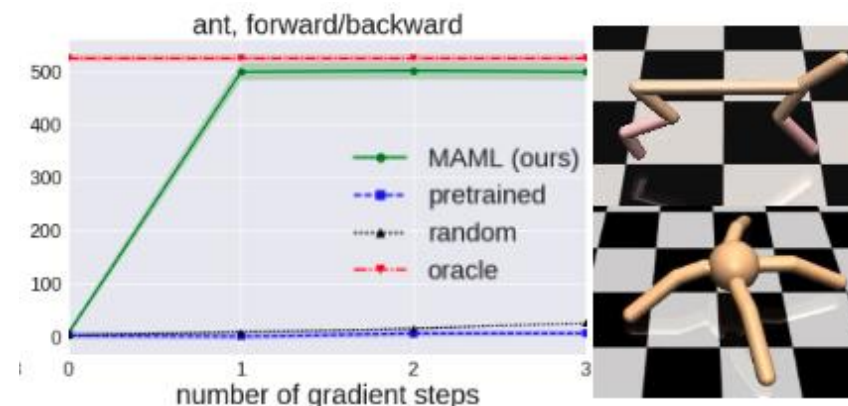
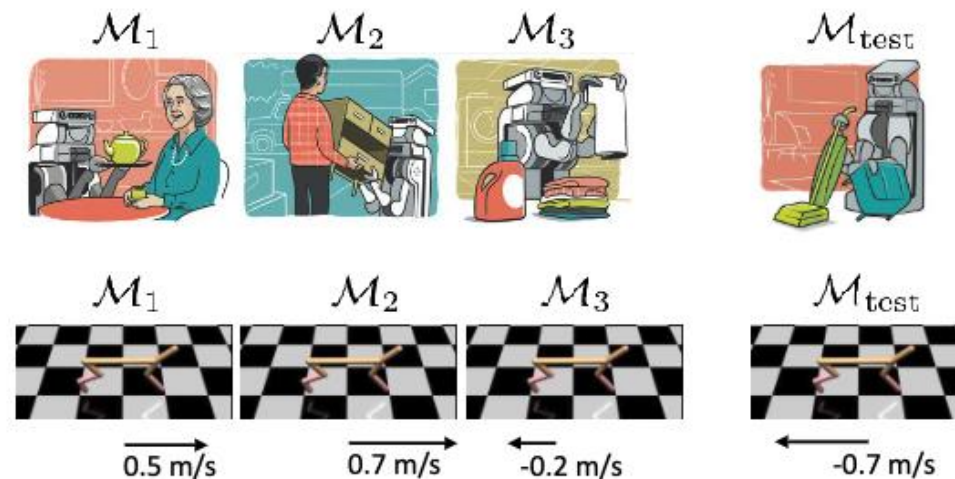
- ❑ “Tasks” become MDPs!
- ❑ Line 6 & 8 become some RL algorithm (e.g., policy gradient)

Algorithm 1 Model-Agnostic Meta-Learning

Require: $p(\mathcal{T})$: distribution over tasks

Require: α, β : step size hyperparameters

- 1: randomly initialize θ
 - 2: **while** not done **do**
 - 3: Sample batch of tasks $\mathcal{T}_i \sim p(\mathcal{T})$
 - 4: **for all** $\mathcal{T}_i$ **do**
 - 5: Evaluate $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$ with respect to K examples
 - 6: Compute adapted parameters with gradient descent: $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta})$
 - 7: **end for** **Note:** the meta-update is using different set of data.
 - 8: Update $\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i})$
 - 9: **end while**
-



Model-based Meta-RL

- ❑ Key idea: apply meta-learning algorithms in model learning!
- ❑ Example 1: MAML to learn the model + sampling-based MPC for control

LEARNING TO ADAPT IN DYNAMIC, REAL-WORLD ENVIRONMENTS THROUGH META-REINFORCEMENT LEARNING

Anusha Nagabandi*, Ignasi Clavera*, Simin Liu,
Ronald S. Fearing, Pieter Abbeel, Sergey Levine, & Chelsea Finn
University of California, Berkeley
{nagaban2, iclavera, simin.liu}@berkeley.edu
{ronf, pabbeel, svlevine, cbfinn}@berkeley.edu

Algorithm 2 Online Model Adaptation (test time)

Require: Meta-learned parameters θ_* , ψ_*

Require: controller(), H , r , n_A

1: $\mathcal{D} \leftarrow \emptyset$

2: **for** each timestep t **do**

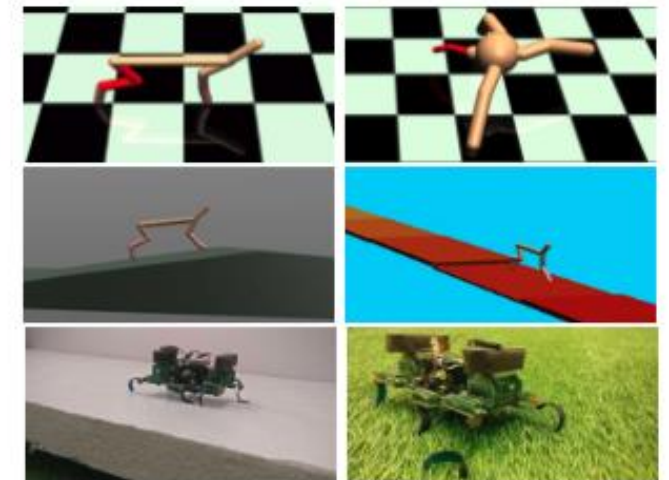
3: $\theta'_* \leftarrow u_{\psi_*}(\mathcal{D}(t-M, t-1), \theta_*)$

4: $\mathbf{a} \leftarrow \text{controller}(\theta'_*, r, H, n_A)$

5: Execute $\mathbf{a}$, add result to $\mathcal{D}$

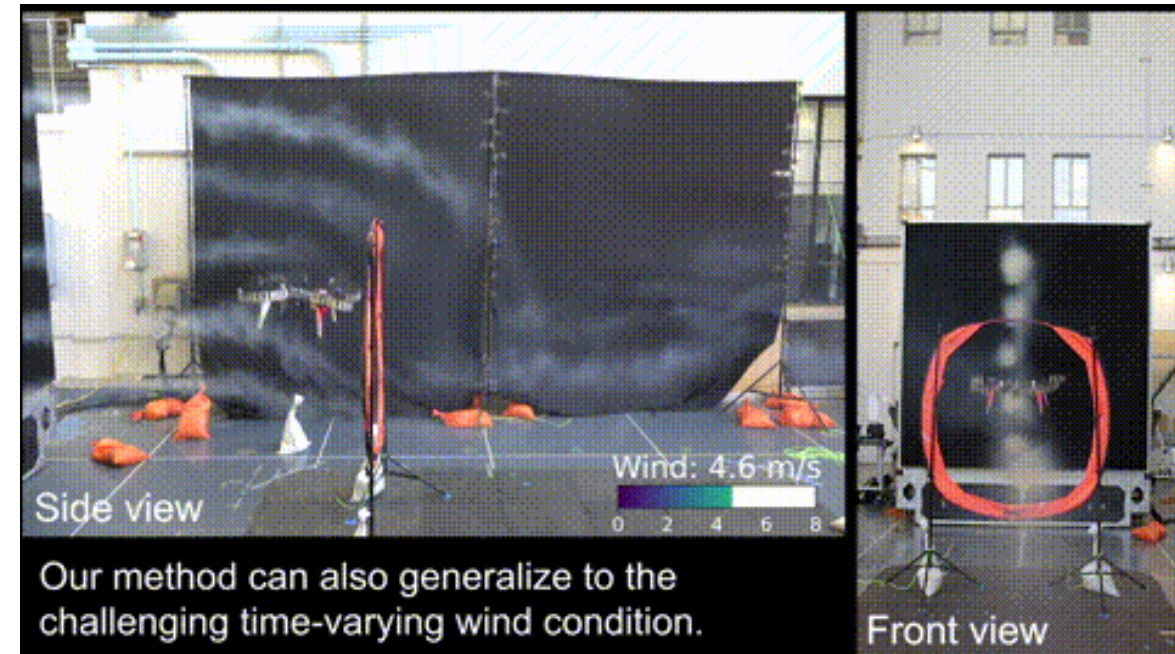
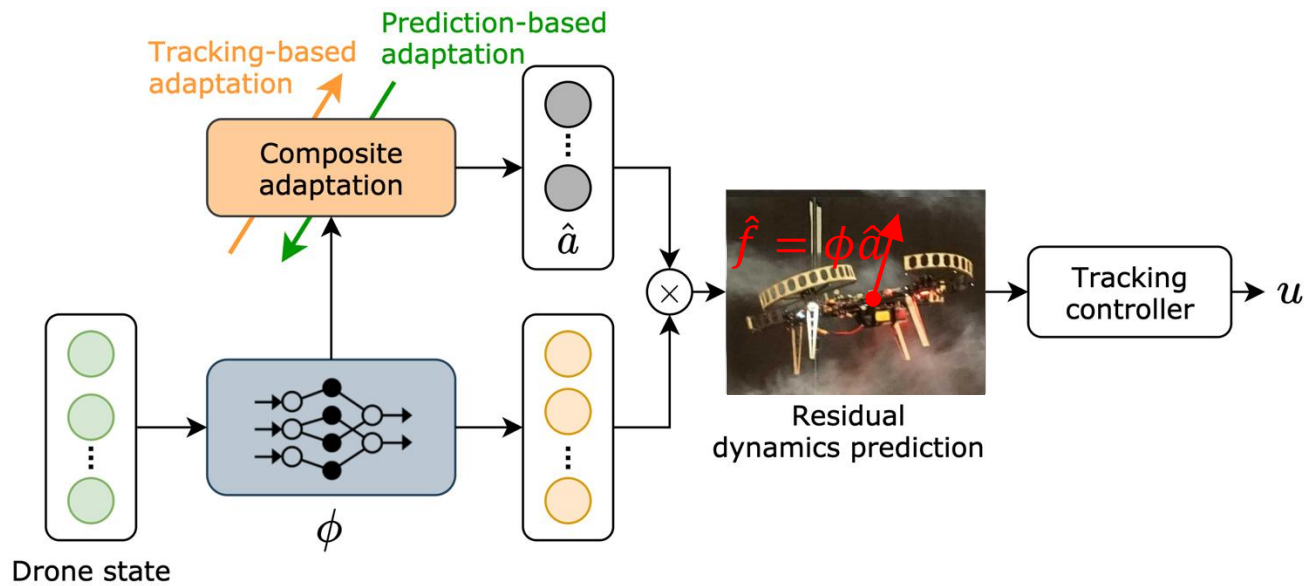
6: **end for**

7: Return rollout $\mathcal{D}$



Model-based Meta-RL

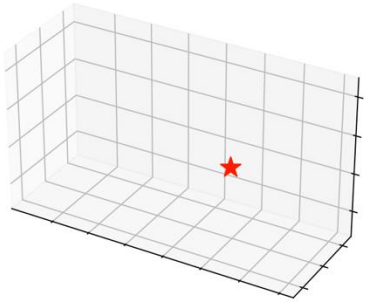
- ❑ Key idea: apply meta-learning algorithms in model learning!
- ❑ Example 2: learn a representation of the dynamics model + adaptive control



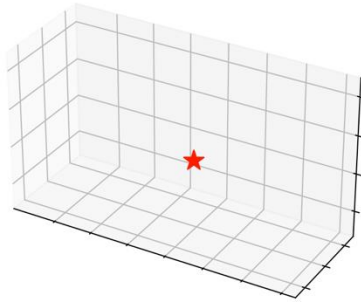
Model-based Meta-RL

- ❑ Key idea: apply meta-learning algorithms in model learning!
- ❑ Example 2: learn a representation of the dynamics model + adaptive control

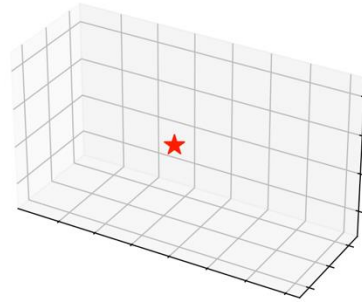
wind speed: 0.0 m/s



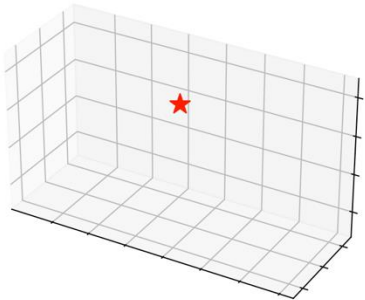
wind speed: 1.3 m/s



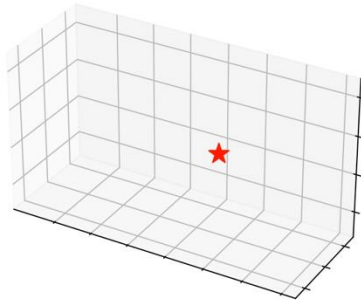
wind speed: 2.5 m/s



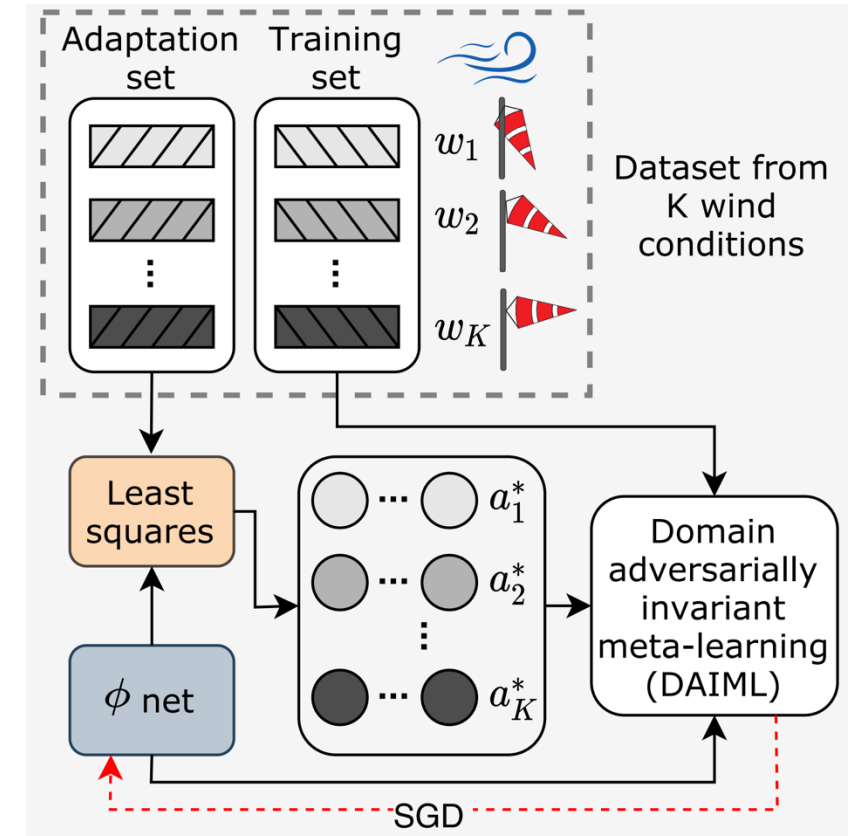
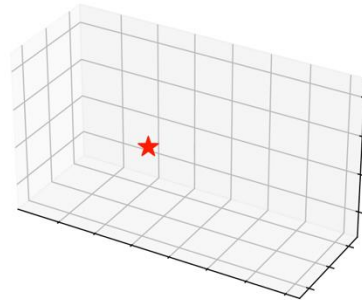
wind speed: 3.7 m/s



wind speed: 4.9 m/s



wind speed: 6.1 m/s



Outline

- ❑ What is multi-task robot learning and adaptive robot learning?
 - Task-wise adaptation
 - Dynamics-wise adaptation

How to adapt?

- ❑ (Classic) adaptive control
- ❑ Meta-learning and meta-RL
- ❑ Policy distillation (teacher-student, etc), end-to-end adaptation
- ❑ Goal-conditioned RL (GCRL)

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

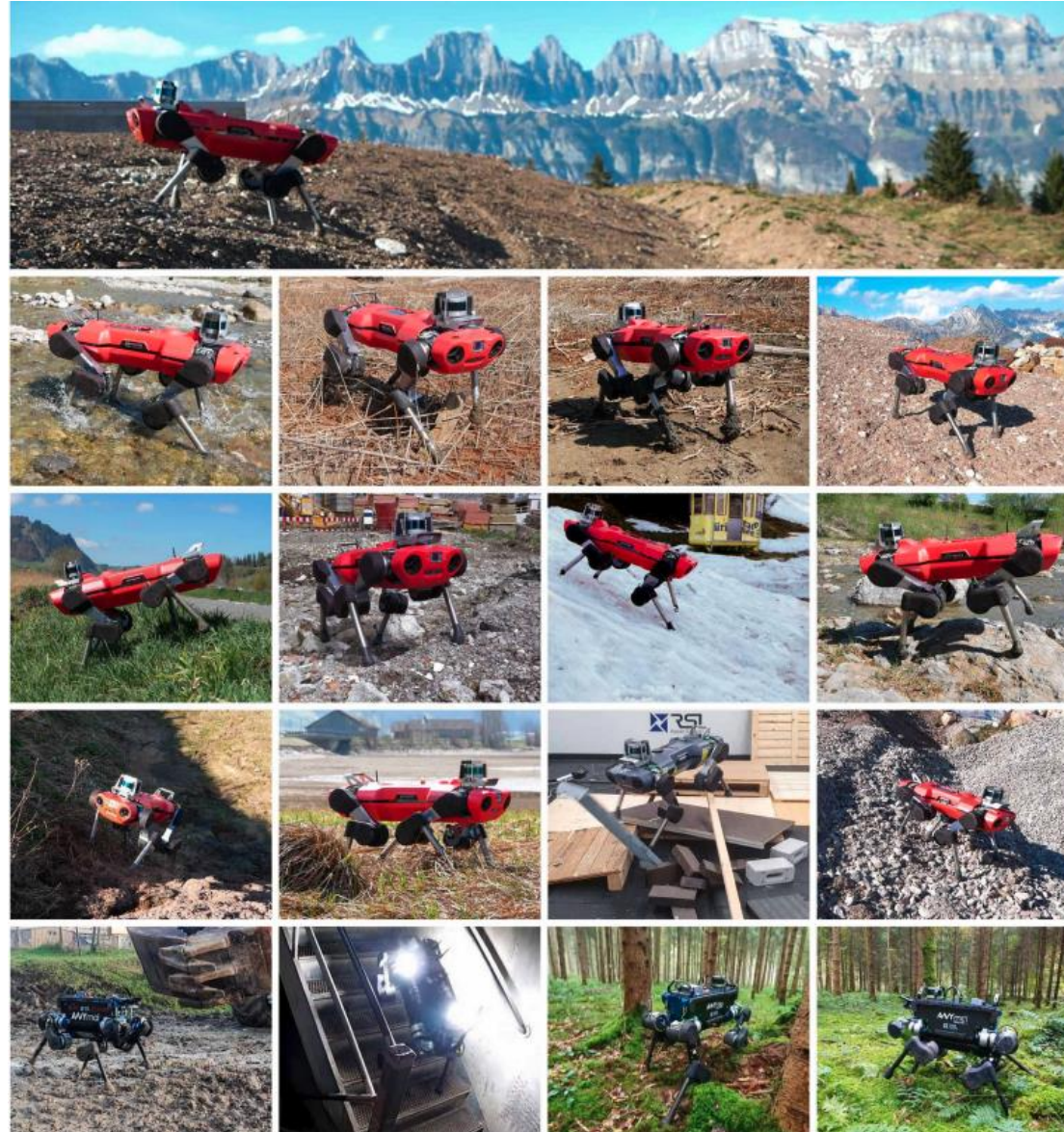
Teacher-Student and Variants

- ❑ *Dynamics-wise* (or *environment-wise*) adaptation
 - Design $a_t = \pi(s_t, z_t)$ and an adaptation law $z_t = A(\dots)$
 - Such that $\pi(s_t, z_t)$ performs well under $s_{t+1} \sim P(\cdot | s_t, a_t, e_t)$

- ❑ Key idea: *learning from a privileged teacher knowing e_t*
 - Simulator: First train a teacher policy with privilege information $\pi(x, e)$
 - Simulator: Then a student policy $\pi_s(x, z = A(\text{available info in the real}))$ learns from $\pi(x, e)$
 - Real: Then deploy $\pi_s(x, z_t = A(\text{available info in the real}))$ in the real

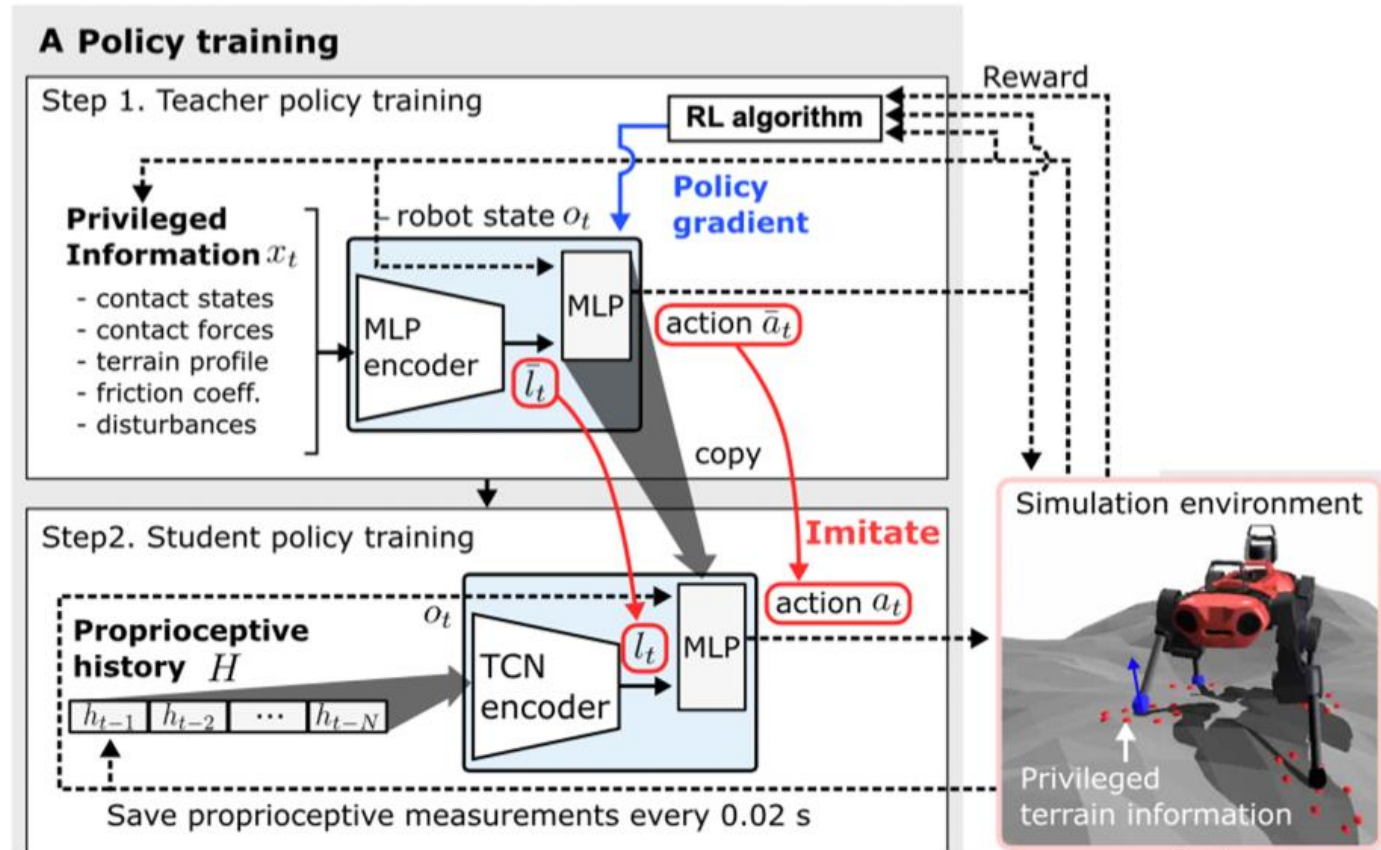
- ❑ Can be viewed as black-box meta-learning
 - 1. Sample task $\mathcal{T}_i$ (or mini batch of tasks)
 - 2. Sample disjoint datasets $\mathcal{D}_i^{\text{tr}}, \mathcal{D}_i^{\text{test}}$ from $\mathcal{D}_i$
 - 3. Compute $\phi_i \leftarrow f_\theta(\mathcal{D}_i^{\text{tr}})$
 - 4. Update θ using $\nabla_\theta \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{test}})$

Learning to Adapt Example: Locomotion



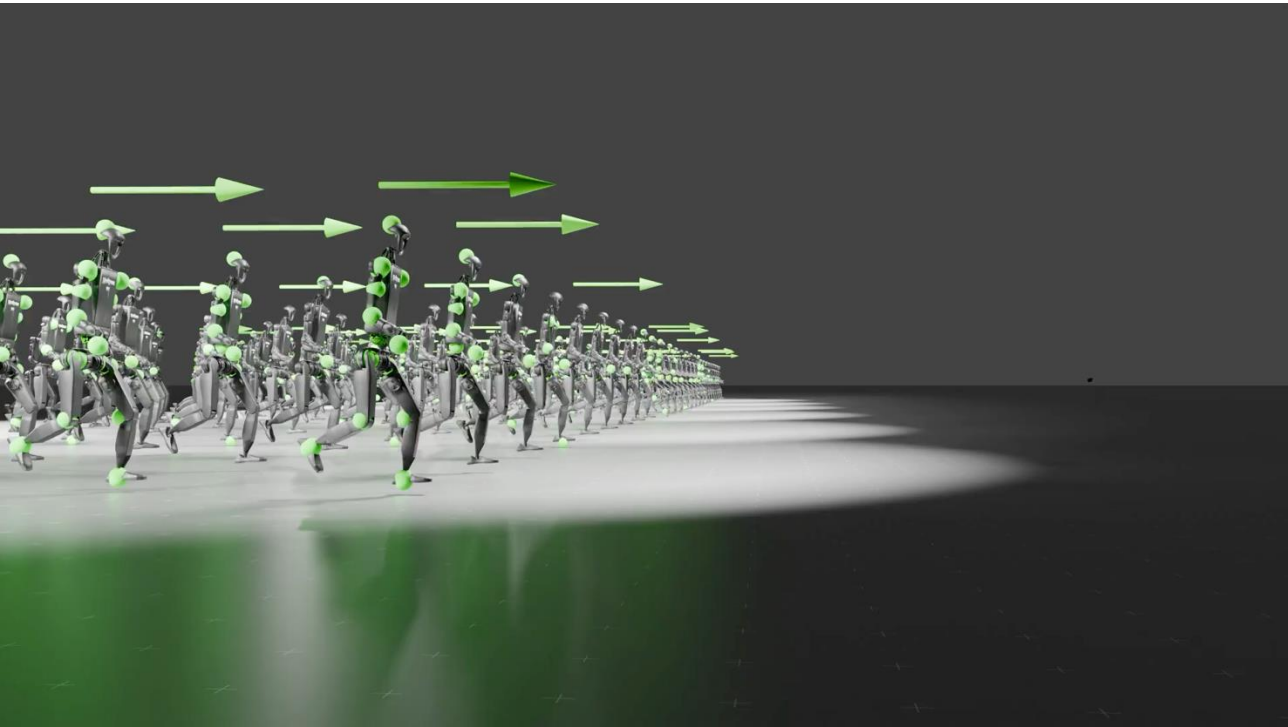
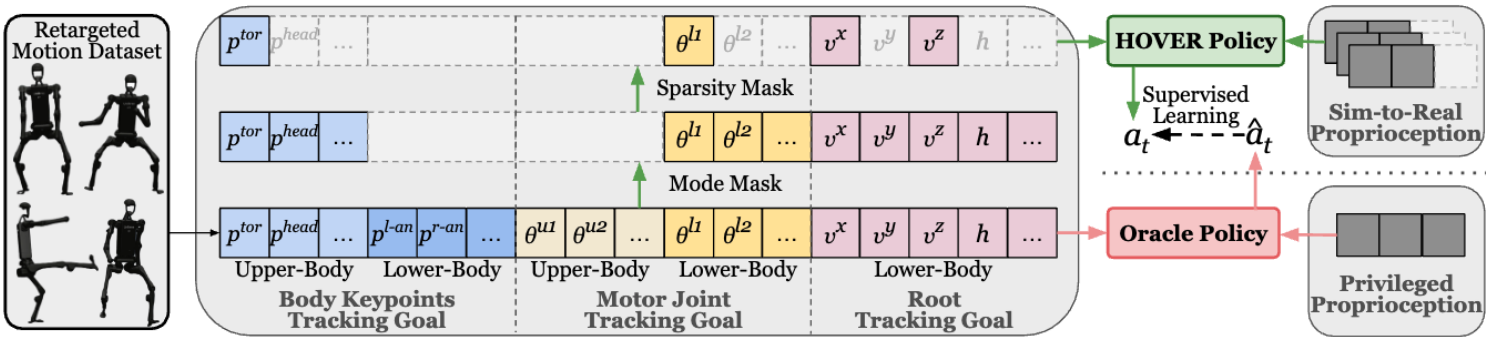
Learning to Adapt Example: Locomotion

- ❑ Privileged information contains almost everything available in sim (contact, terrain, friction, disturbance, etc)
- ❑ Students only learn from proprioceptive history (IMU, joint angle, etc)
- ❑ Privileged teacher is trained using RL (PPO, different from the driving example)



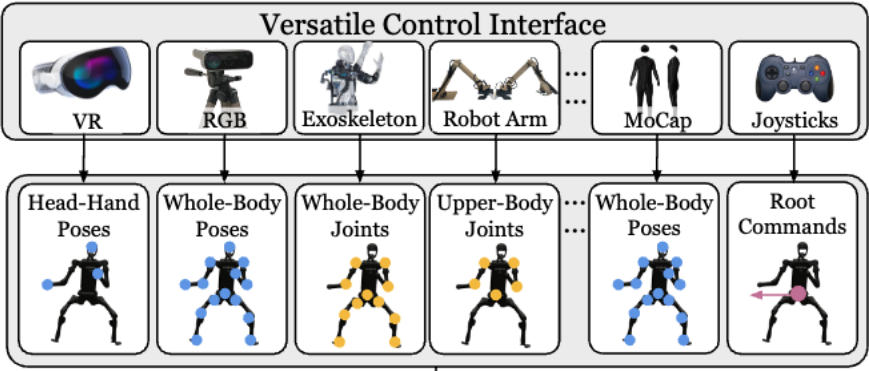
Learning to Adapt Example: Humanoids

❑ Large-scale distillation



HOVER: Versatile Neural Whole-Body Controller for Humanoid Robots

Tairan He^{*1,2} Wenli Xiao^{*1,2} Toru Lin^{1,3} Zhengyi Luo^{1,2} Zhenjia Xu¹ Zhenyu Jiang^{1,4}
Jan Kautz¹ Changliu Liu² Guanya Shi² Xiaolong Wang^{1,5} Linxi “Jim” Fan^{†1} Yuke Zhu^{†1,4}



Some Variants: Student Learning Not in Action Space

- ❑ Student learning doesn't have to happen in the action space
 - RMA: in latent space

RMA: Rapid Motor Adaptation for Legged Robots

Ashish Kumar
UC Berkeley

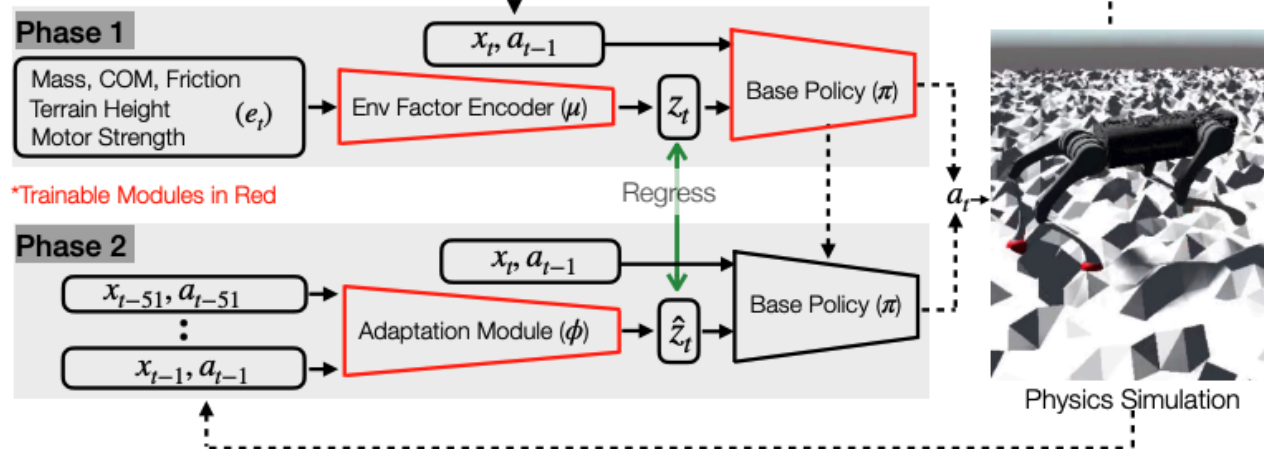
Zipeng Fu
Carnegie Mellon University

Deepak Pathak
Carnegie Mellon University

Jitendra Malik
UC Berkeley, Facebook

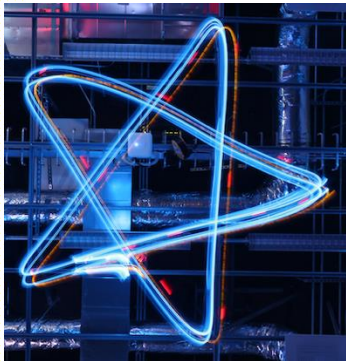
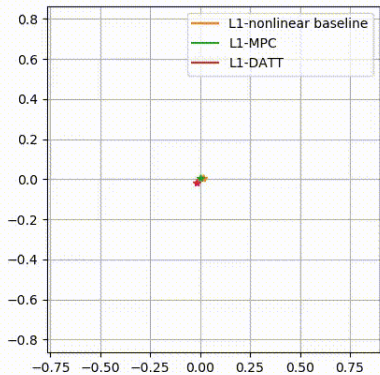


A) Training in Simulation

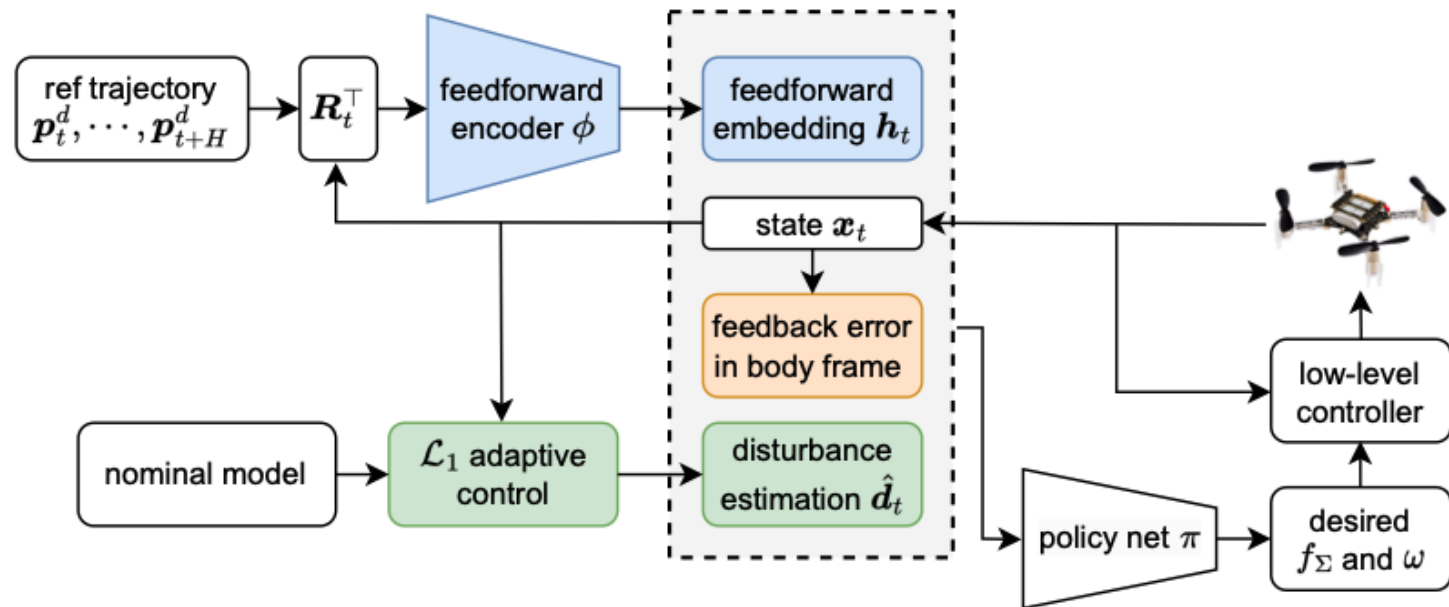


Some Variants: Student Learning Not in Action Space

- ❑ Student learning doesn't have to happen in the action space
 - DATT: student predicts unknown dynamics e directly



$$\begin{aligned}\dot{\mathbf{p}} &= \mathbf{v} \\ m\dot{\mathbf{v}} &= m\mathbf{g} + \mathbf{R}\mathbf{e}_3 f_\Sigma + \mathbf{d} \\ \dot{\mathbf{R}} &= \mathbf{R}S(\boldsymbol{\omega}) \\ \mathbf{J}\dot{\boldsymbol{\omega}} &= \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}\end{aligned}$$



End-to-End Adaptation

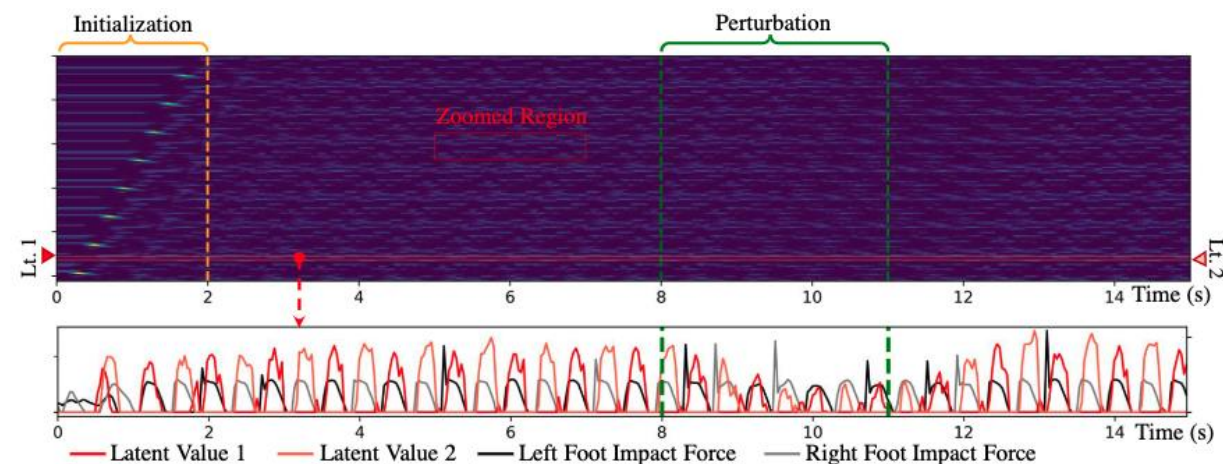
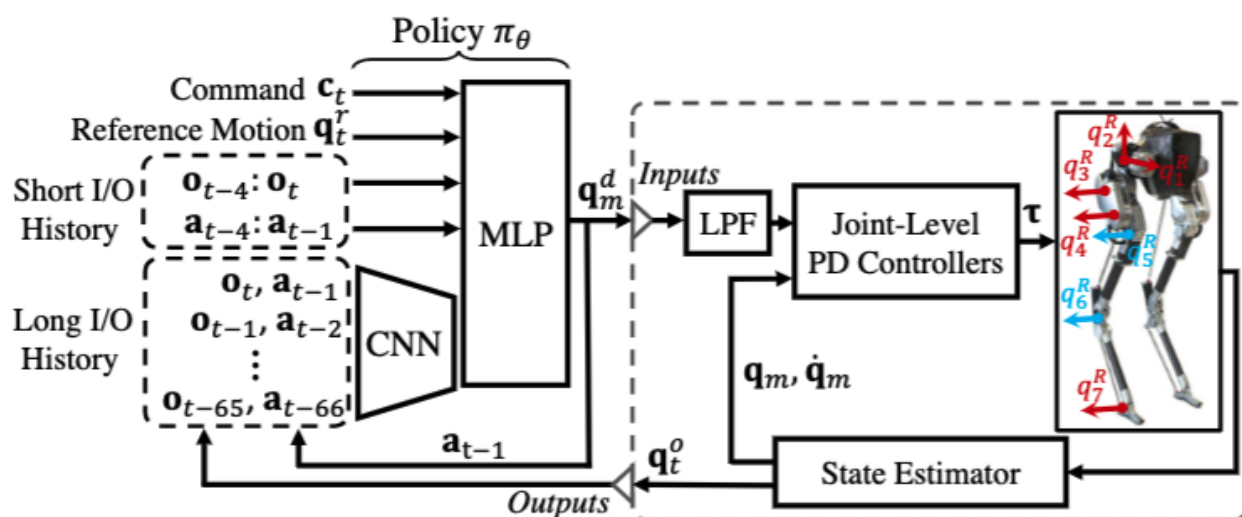


End-to-End Adaptation

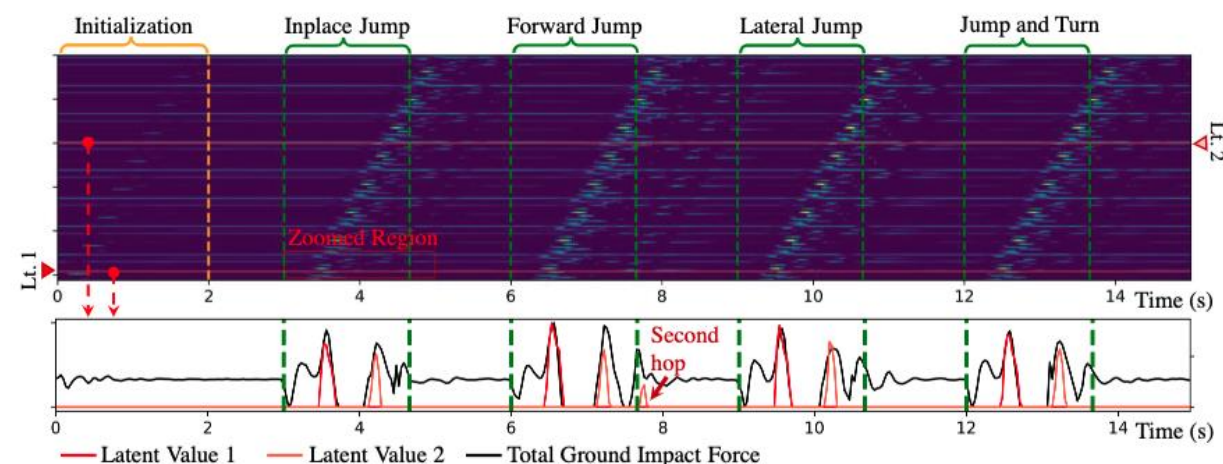
Reinforcement Learning for Versatile, Dynamic, and Robust Bipedal Locomotion Control

Zhongyu Li¹, Xue Bin Peng², Pieter Abbeel¹, Sergey Levine¹, Glen Berseth^{3,4}, Koushil Sreenath¹

¹University of California Berkeley ²Simon Fraser University ³Université de Montréal ⁴Mila – Quebec AI Institute



(a) (Top) Recorded latent representation after long-term I/O history encoder during running. (Bottom) Comparison of two selected dimensions (marked as red lines in the top plot) with recorded impact forces on each of the robot's feet.



(c) (Top) Recorded latent representation after long-term I/O history encoder during jumping. (Bottom) Comparison of two selected dimensions (marked as red lines in the top plot) with recorded total impact forces on both of the robot's two feet.

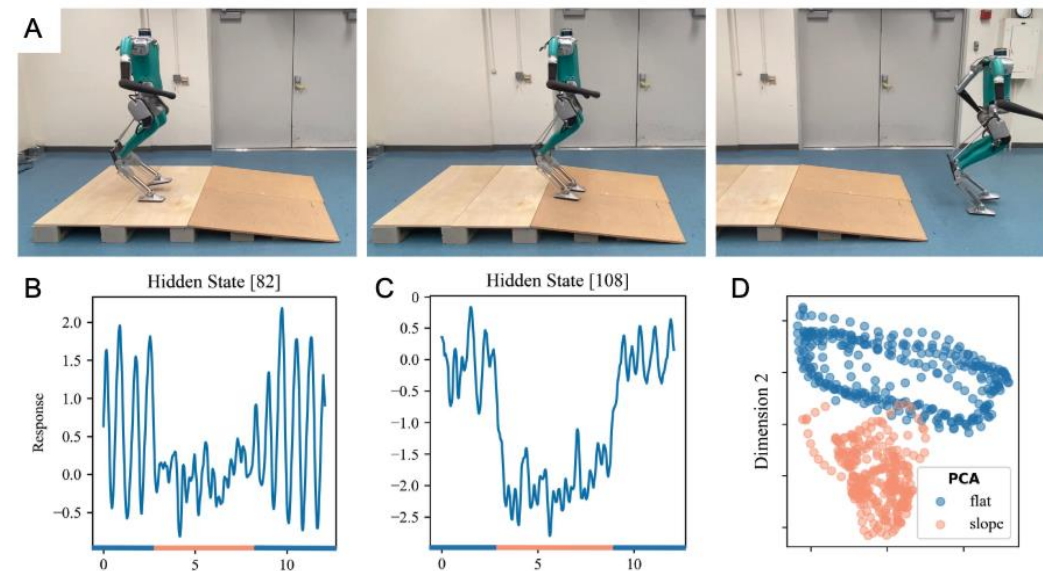
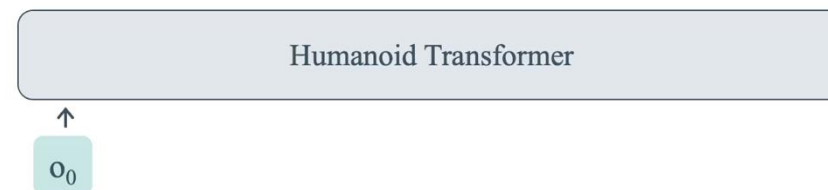
End-to-End Adaptation

Learning Humanoid Locomotion with Transformers

Ilija Radosavovic^{*} Tete Xiao^{*} Bike Zhang^{*} Trevor Darrell[†] Jitendra Malik[†] Koushil Sreenath[†]

Authors contributed equally and listed in alphabetical order

University of California, Berkeley



Outline

- ❑ What is multi-task robot learning and adaptive robot learning?
 - Task-wise adaptation
 - Dynamics-wise adaptation

How to adapt?

- ❑ (Classic) adaptive control
- ❑ Meta-learning and meta-RL
- ❑ Policy distillation (teacher-student, etc), end-to-end adaptation
- ❑ Goal-conditioned RL (GCRL)

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Goal-Conditioned RL (GCRL)

- ❑ Objective: get an adaptive policy $a_t = \pi(s_t, g)$
 - $\pi(s_t, g)$ maximizes cumulative reward $r(s_t, a_t, g)$
 - Common issues: overfitting, gradient conflicts, etc

Goal-Conditioned Reinforcement Learning: Problems and Solutions

- ❑ Survey paper:

Minghuan Liu*, Menghui Zhu*, Weinan Zhang[†]
Shanghai Jiao Tong University
{minghuanliu, zerozmi7, wnzhang}@sjtu.edu.cn

- ❑ Example algorithm:

Learning to Reach Goals via Iterated Supervised Learning

Dibya Ghosh*
UC Berkeley

Abhishek Gupta*
UC Berkeley

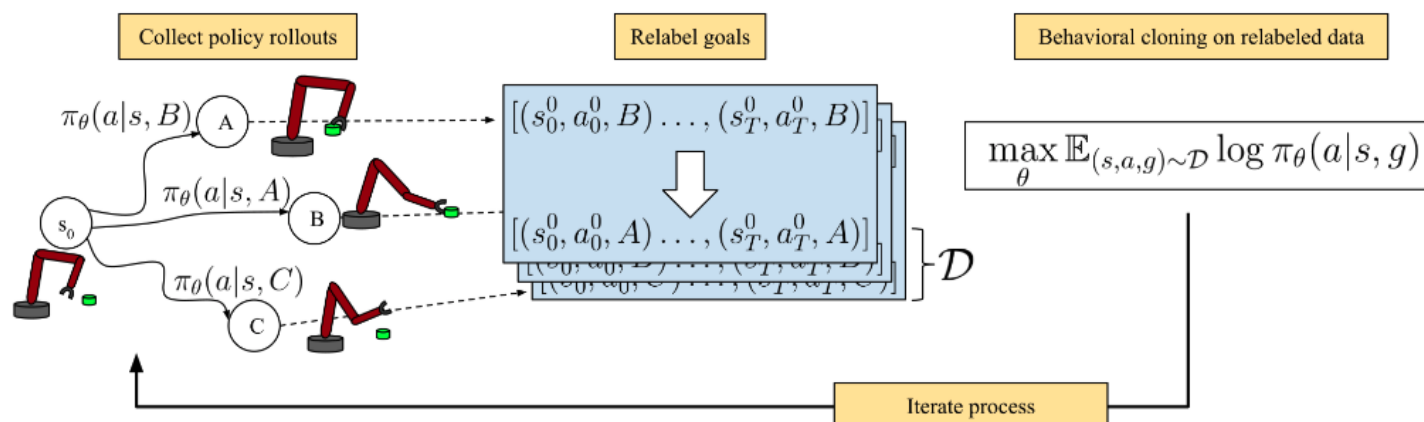
Ashwin Reddy
UC Berkeley

Justin Fu
UC Berkeley

Coline Devin
UC Berkeley

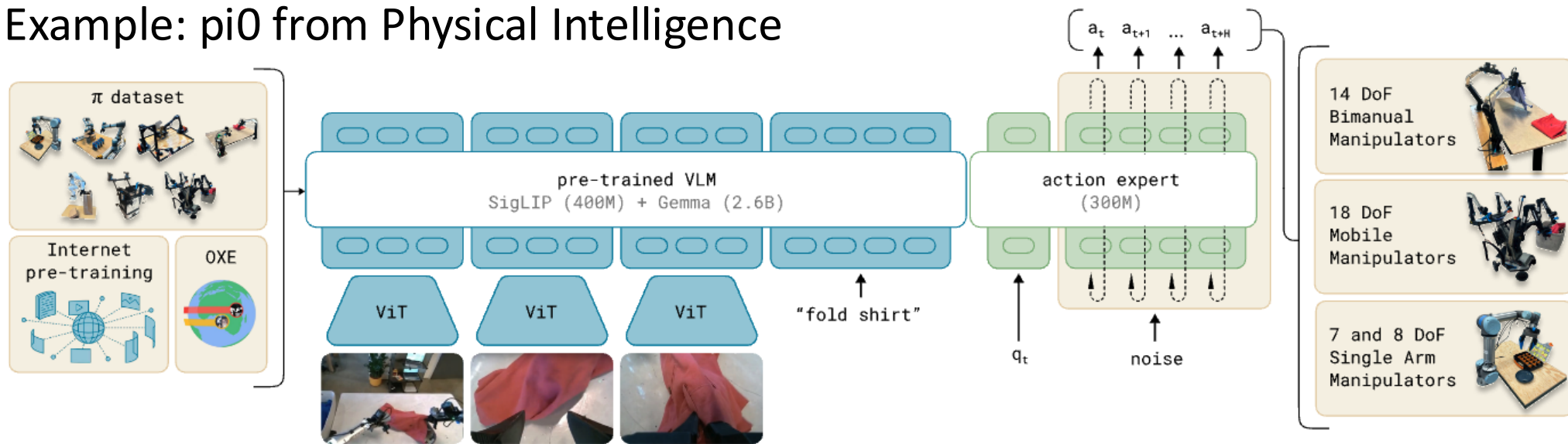
Benjamin Eysenbach
Carnegie Mellon University

Sergey Levine
UC Berkeley



Vision-Language-Action Generalist Policy

- ❑ Multi-task & “goal” conditioned
- ❑ Example: pi0 from Physical Intelligence



π_0 : A Vision-Language-Action Flow Model for General Robot Control

Physical Intelligence

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, Ury Zhilinsky
<https://physicalintelligence.company/blog/pi0>



Thank You!